HE-IFD: Privacy-Preserving One-Shot Federated Fine-Tuning of Pretrained Models under Homomorphic Encryption

Halil İbrahim Kanpak, Sinem Sav, Alptekin Küpçü

***Abstract*—Federated learning lets parties train a shared model without pooling their data, but its one-shot variant, which communicates in a single round, faces an unresolved privacy dilemma: sending a client's contribution in plaintext exposes it, while protecting it with differential privacy injects noise that degrades the released model and offers only a statistical, not a cryptographic, guarantee, and homomorphic encryption has so far been confined to multi-round encrypted training or to secure-aggregation primitives. We present HE-IFD, a one-shot federated fine-tuning whose contributions are protected by homomorphic encryption. Each client fine-tunes a small adapter on a frozen pretrained backbone for a bounded number of steps, and the server combines the resulting parameter displacements with a single sample-weighted linear operation under multiparty CKKS, at multiplicative depth one. The key to making a one-shot linear aggregation work under heterogeneous data is a shared frame, which a frozen pretrained backbone supplies for free: every client fine-tunes from the same public initialization, so there is no alignment phase and nothing data-derived leaves a client before encryption. Across vision and language tasks on frozen pretrained backbones, the method produces a common model stronger than any client's own and, outside the most skewed regimes, close to a centralized one, while giving each client coverage of classes it never observed locally; the residual gap to centralized accuracy is set by how much of the label space each client sees and narrows as that coverage grows. A real multiparty CKKS implementation reproduces the plaintext aggregate within the approximation bounds of CKKS at tens of megabytes per round.**

***Index Terms*—Federated learning, one-shot federated learning, federated fine-tuning, homomorphic encryption, multiparty computation, privacy-preserving machine learning.**

I. Introduction

Many organizations would benefit from training a shared model on their combined data but cannot pool it, whether for legal, competitive, or privacy reasons. Federated learning addresses this by exchanging model updates instead of data, but in its usual form it does so over many rounds of communication, and each round discloses a fresh update computed on private data. One-shot federated learning removes the first cost by collapsing the entire interaction into a single upload and download. It does not, by itself, remove the second. A client's single contribution still encodes its private data, and the field's answers to this have been unsatisfying at the extremes: sending the contribution in plaintext exposes it, while protecting it with differential privacy injects noise into the released model and provides only a statistical, not a cryptographic, guarantee. Homomorphic encryption promises a way out by operating on contributions while they stay encrypted, but it has been applied to federated learning either as multi-round encrypted training, whose cost is measured in hours and gigabytes, or as a secure-aggregation primitive embedded in an iterative protocol. No existing method delivers a one-shot federated distillation whose contributions are protected cryptographically rather than by accuracy-degrading noise.

We present **HE-IFD**. Each client fine-tunes a small low-rank adapter and classifier head on a frozen public backbone, starting from a shared public initialization, and the server combines the clients' contributions with a single linear operation, $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$, evaluated under multiparty CKKS. The server holds the shared initialization θ_0 as a public constant and never modifies it during the protocol: it only adds the aggregated displacement to this preserved model, so no training takes place on the server side. Because the combination is linear, it maps onto the operations a homomorphic scheme evaluates at multiplicative depth one, without bootstrapping, and moves tens of megabytes per round rather than hundreds of gigabytes. The privacy of the contributions is therefore cryptographic, and because the backbone is frozen and never enters the encrypted combination, the same protocol and the same cost carry across pretrained backbones in vision and language.

The difficulty a one-shot linear aggregation must overcome is that independently trained models do not average well under heterogeneous data: starting from different initializations and fitting different local distributions, their updates point in conflicting directions and cancel when summed. Fine-tuning a frozen pretrained backbone removes this difficulty without any alignment phase. The fixed backbone places every client in a common representation, and every client starts its small trainable unit from the same public initialization, so their bounded updates are expressed in one frame and reinforce rather than cancel. Where prior one-shot federated learning must first agree on a starting point by exchanging per-class summaries (an extra round that exposes data-derived quantities and, when protected, spends a differential-privacy budget), a pretrained backbone supplies the shared frame without that step, and nothing data-derived leaves a client before encryption.

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

We show across vision and language tasks that the method produces a common model stronger than any client's own and, outside the most skewed regimes, close to a centralized model trained on the pooled data, while every client gains the ability to classify categories it never observed locally. We confirm in a real multiparty CKKS implementation that the encrypted aggregation reproduces the plaintext result within the approximation bounds of CKKS at tens of megabytes per round, and we measure with membership inference that the only quantity exposed in the clear, the released model, leaks little about any client's data.

Our contributions are as follows.

- We introduce, to our knowledge, the first one-shot federated fine-tuning whose contributions are protected by homomorphic encryption, with a single server operation of multiplicative depth one under multiparty CKKS (section III).
- We show that fine-tuning a frozen pretrained backbone supplies the shared frame a one-shot linear aggregation needs at no cost, eliminating the data-dependent alignment phase that prior one-shot federated learning requires, and with it the leakage and the differential-privacy budget that phase entails (sections III and IV-B).
- We give a privacy design in which the contributions are protected by cryptography alone, with no perturbation and no differential privacy, so the only quantity exposed in the clear is the final shared model, whose residual leakage we measure directly with membership inference (section IV-G).
- We demonstrate that the method yields a common, superior model across frozen pretrained backbones in both modalities, and validate the encrypted aggregation in a real multiparty CKKS implementation at a cost of tens of megabytes per round (sections IV-B and IV-F).

II. Related Work

The setting we study lies at the intersection of three lines of work that have largely developed in isolation: how federated learning communicates, how it is made private, and how it has been compressed into a single round. Tracing how each arrived at its present state makes clear both what is established and where the gap we address lies.

Federated learning began as an iterative procedure. In the original formulation a server repeatedly broadcasts a global model, clients improve it on their local data, and the server averages the returned updates [1]. This works well but inherits two persistent costs from its iterative nature. The first is communication: reaching a good model can take tens or hundreds of rounds, each moving a full model in both directions. The second is exposure: every round reveals a fresh update computed on private data, and a long sequence of such updates is a rich target for inference. Much of the subsequent literature can be read as attempts to control one of these two costs without worsening the other.

The exposure problem has been met with two broad tools, distinguished by whether they alter the model that is ultimately released. Secure aggregation lets a server learn only the sum of the clients' updates and nothing about any individual one, using masking protocols that cancel in the aggregate [2]; it is lossless, in that the aggregate is exact, but it protects only the per-round sum and still operates within the iterative protocol. Differential privacy instead perturbs the updates themselves, most commonly through differentially private stochastic gradient descent [3]; the guarantee is strong and composable, but because the noise enters the released model it trades privacy directly against accuracy. This distinction, between privacy that costs accuracy and privacy that does not, is the axis along which our own privacy design is best understood, and it recurs throughout what follows.

Homomorphic encryption offers a third route, protecting the updates by computing on them while they remain encrypted. In its most ambitious form it has been used to train a neural network entirely under encryption across many parties. POSEIDON, for instance, runs encrypted stochastic gradient descent under multiparty CKKS, with the nonlinear activations replaced by polynomial approximations so that they can be evaluated homomorphically [4]. The privacy adds no accuracy cost and the result is strong, but the approach remains fundamentally iterative and pays for the encrypted nonlinearities with a deep multiplicative circuit, repeated bootstrapping, and a wall-clock measured in hours. A lighter line keeps the encryption but narrows its job to aggregation alone: schemes that combine multi-key CKKS with masking reduce a round of secure summation to a single non-interactive client-to-server transmission [5]. These are efficient and genuinely cryptographic, but they encrypt the sum of model updates within an iterative training protocol rather than a one-shot contribution, and they perform aggregation rather than knowledge transfer. On the encryption axis these works are the closest to ours, and the contrast is precise: they are either multi-round or are secure-aggregation primitives, whereas we encrypt a single distillation displacement.

Homomorphic encryption has also been applied to federated learning purely for aggregation, encrypting the per-round updates so the server can sum them without seeing any in the clear; BatchCrypt, FedML-HE, and FedSHE develop this for cross-silo training with various packing and quantization optimisations [6], [7], [8]. These remain multi-round and, like secure aggregation, protect the sum rather than transferring knowledge. A recurring obstacle across all encrypted-training work is the nonlinearity: activations must be replaced by polynomial approximations to be evaluated homomorphically, which deepens the multiplicative circuit and is delicate to train, as a line of work on polynomial activations and encryption-friendly networks documents [9], [10], [11], [12], [13]. Our design sidesteps this entirely: the only encrypted operation is linear, so no activation is ever evaluated under encryption and the circuit stays at depth one.

The communication problem has driven the parallel development of one-shot federated learning, which collapses the entire exchange into a single upload and download [14], [15]. The earliest approaches transferred knowledge rather than weights, having clients exchange predictions on a shared public set so that heterogeneous models could teach one another [16], [17]; in practice these remained iterative, repeating

the exchange to converge. Ensemble distillation made the distillation step explicit by training a server model against the clients' aggregated predictions, though in its standard configuration it too runs for many rounds [18]. The decisive step to a true single round came with data-free one-shot methods, which avoid any shared data by having the server distil the clients' models through a generator or through synthesised inputs, as in DENSE and its successors [19], [20], and with fusion-based methods that stitch client models together with lightweight learned adapters [21] or aggregate them through a layer-wise posterior [22], [23]. These methods are the closest to ours on the one-shot and distillation axes, and they establish that a single round can produce a usable joint model. What they do not provide is any cryptographic protection: they operate in plaintext, so the contributions each client sends are exposed.

Closing that last gap, a handful of works add privacy to one-shot federated learning, and they do so with differential privacy. FedAUXfdp distils through a pretrained feature extractor on auxiliary public data and releases the client contributions under a differentially private mechanism [24]; related approaches protect a diffusion-generated [25] or a teacher-ensemble [26] surrogate, or add noise-free differential privacy to the distillation itself [27], under the same kind of guarantee. These are the natural quantitative peers for our method, since they share its one-shot, distillation-based structure, but their privacy is lossy in exactly the sense above: the noise needed for a meaningful budget is added to the artifact that is released, and accuracy falls as the budget tightens.

The reason privacy is the binding concern here is that the artifacts federated protocols exchange leak. Sharing gradients or model updates permits reconstruction of training inputs [28], [29], [30] and supports membership and property inference [31], [32], [33], [34], [35], [36], and even released confidences enable model inversion [37], [38]. Federated distillation, which shares predictions or a public-data signal instead of gradients, was hoped to sidestep this, but a growing body of attacks shows it leaks as well: paired-logit inversion recovers images from shared logits [39], and membership can be inferred from distillation outputs and from public-dataset usage [40], [41], [42], [43], [44]. This is why we protect the contributions cryptographically rather than by perturbation, and why we measure in section IV-G what the released model still reveals.

Seen together, the two strongest lines never meet. The cryptographic work that would protect contributions without loss is either multi-round encrypted training or a secure-aggregation primitive, and the one-shot distillation work that communicates in a single round is either plaintext or made private only by lossy perturbation. The method we present occupies the empty intersection: a one-shot federated distillation whose single encrypted operation is a linear aggregation under multiparty homomorphic encryption, giving the accuracy-preserving privacy of the cryptographic line at the communication budget of the one-shot line.

III. Method

A. Overview

We consider N clients that hold private labelled data over a common label space of C classes and want a shared classifier without pooling their data. Client j holds $\mathcal{D}_j$ of size n_j; the data are heterogeneous, with each client's class proportions drawn from a Dirichlet distribution of concentration α (smaller α means more skew). Two constraints separate our setting from ordinary federated learning. The protocol is *one-shot*: one upload and one download per client, with no iterative exchange. And privacy is *cryptographic*: a client's contribution is never revealed in the clear, and we are unwilling to accept the accuracy loss that differential privacy incurs when it is the only line of defence.

These two constraints largely determine the design. Under encryption the server is blind: it operates on ciphertexts that are by construction indistinguishable from random, so it cannot read a value, compare two values, or branch on the data, and every multiplication it performs spends part of a finite noise budget. The only computation it can afford at scale is a shallow, fixed, linear one (a weighted sum), and a weighted sum of independently trained models is meaningful only when those models already share a frame. A from-scratch federation has no such frame, and building one would take either several rounds or server-side computation on the models themselves, neither of which encryption allows. A *pretrained* model supplies the frame directly: freezing a public backbone fixes the representation every client sees, so the small part each client trains starts from the same place and stays close. Fine-tuning a pretrained model is therefore not a convenience but the learning setting this protocol admits, and it removes the need for any alignment step before training, the property that most distinguishes HE-IFD from prior one-shot federated learning, which spends an extra communication phase, and often a differential-privacy budget, to bring clients into a common frame before they begin.

Concretely, every client uses a frozen public backbone φ followed by a small trainable unit: low-rank adapters together with a classifier head, with parameters ψ. The backbone never changes, is never transmitted, and is never encrypted; only the trainable unit is fine-tuned, communicated, and operated on under encryption. Keeping that unit small is what makes the encrypted step inexpensive: it is a tiny fraction of the backbone (a low-rank adapter and head, on the order of 10^5 parameters against a backbone of 10^8), so a client's entire encrypted contribution is a few tens of ciphertexts rather than the model itself. Every client initializes the trainable unit to the *same* public constant θ_0, the standard adapter initialization, agreed in advance and carrying no client data. There is no prototype exchange and no peer-to-peer phase; nothing data-derived ever leaves a client before encryption.

The protocol is two steps, shown in fig. 1. Starting from the shared initialization θ_0, each client fine-tunes its trainable unit on its own data for a bounded number of steps and records the displacement it travelled, $\Delta_j = \theta_j^{(K)} - \theta_0$. The server then combines the encrypted displacements into a single global model, $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$, using only homomorphic

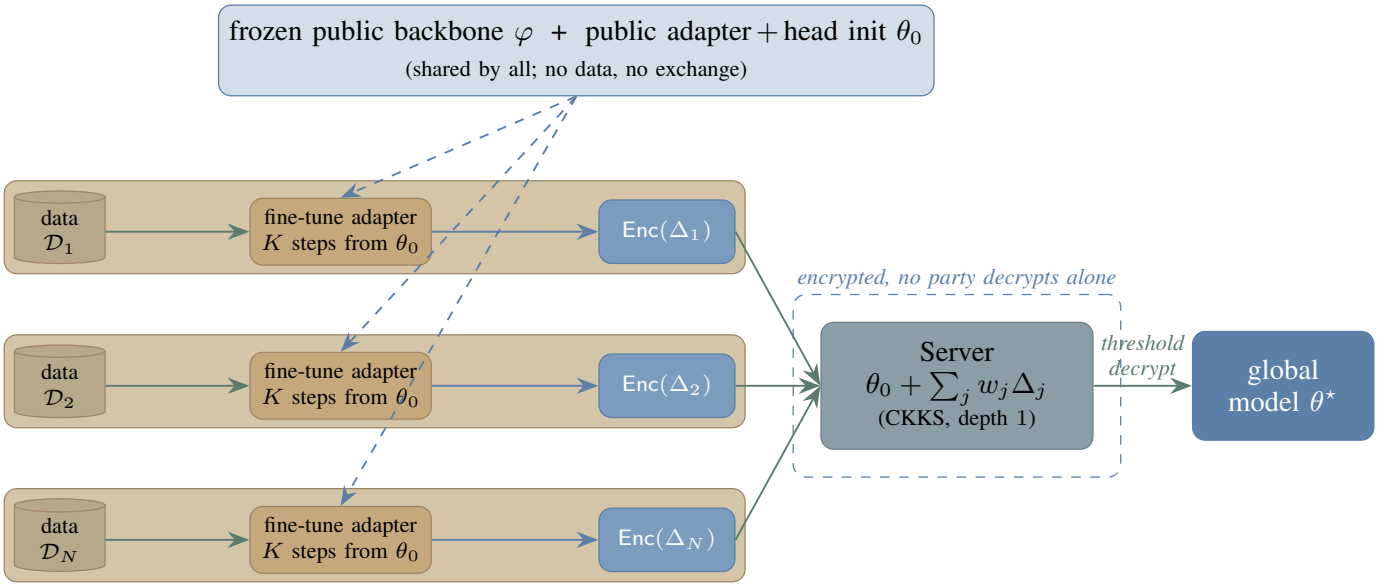

Fig. 1. HE-IFD. A frozen public backbone φ and a public initialization θ_0 of the small trainable unit (low-rank adapter and head) are shared by all clients and carry no data, so there is no alignment phase; the backbone is never trained or encrypted. Each client fine-tunes the adapter on its own private data $\mathcal{D}_j$ for a bounded number of steps from θ_0 and encrypts the resulting displacement $\Delta_j = \theta_j - \theta_0$. The server's sole operation is the sample-weighted linear combination $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$ under multiparty CKKS (additions and public-scalar multiplications only; multiplicative depth one); it never decrypts. A threshold of clients jointly decrypts the global model $\theta^\star$.

Algorithm 1 One-shot federated fine-tuning with encrypted aggregation. $\mathsf{Enc}(\cdot)$ denotes encryption under the clients' collective CKKS key.

Require: clients $1, \dots, N$ with data $\mathcal{D}_j$; frozen backbone φ; public init θ_0; steps K

clients run distributed key generation, yielding a collective CKKS key

Local fine-tuning (each client j in parallel)

$\theta_j^{(0)} \leftarrow \theta_0$

for $k = 1$ to K **do**

  $\theta_j^{(k)} \leftarrow \theta_j^{(k-1)} - \eta \nabla \mathcal{L}\big(f_\theta; \mathcal{D}_j\big)$ ▷ supervised fine-tuning of the trainable unit

end for

$\Delta_j \leftarrow \theta_j^{(K)} - \theta_0$; upload $\mathsf{Enc}(\Delta_j)$

Encrypted aggregation (server)

$\mathsf{Enc}(\theta^\star) \leftarrow \theta_0 + \sum_j w_j \cdot \mathsf{Enc}(\Delta_j)$ ▷ PT×CT, CT+CT; depth 1

broadcast $\mathsf{Enc}(\theta^\star)$

Threshold decryption (clients)

a threshold of clients jointly decrypt $\mathsf{Enc}(\theta^\star)$ to recover $\theta^\star$

additions and multiplications by public scalars, after which a threshold of clients jointly decrypts the result.

What we are after is not a model that beats each client on that client's own dominant classes; a local specialist on a narrow slice of the label space is difficult to surpass on that slice, and we do not claim to. We are after a common model every party can use, that classifies well across the whole label space including classes a given client never observed locally, and that is produced without any party exposing its data or its update in the clear.

TABLE I
NOTATION.

Symbol	Meaning
N	number of participating clients
$\mathcal{D}_j,\ n_j$	client j's local dataset and its size $n_j = \lvert\mathcal{D}_j\rvert$
C	number of classes
α	Dirichlet concentration (label heterogeneity)
φ	frozen public backbone (never trained or encrypted)
ψ	parameters of the trainable unit (adapter + head)
f_θ	model on the frozen features, parameters θ
θ_0	shared public initialization of the trainable unit
K	local fine-tuning steps
Δ_j	displacement $\theta_j^{(K)} - \theta_0$
w_j	sample weight $n_j / \sum_i n_i$
$\theta^\star$	aggregated global model $\theta_0 + \sum_j w_j \Delta_j$

B. Notation

Table I collects the symbols used throughout. Client indices are $j \in \{1, \dots, N\}$.

C. Multiparty Homomorphic Encryption

We build on the CKKS scheme [45], a ring-learning-with-errors construction [46] that performs approximate arithmetic on vectors of real numbers under encryption. CKKS packs many real slots into one ciphertext and supports two homomorphic operations of interest here: addition of two ciphertexts, and multiplication of a ciphertext by a plaintext. It is a levelled scheme, meaning each multiplication consumes one level of a finite budget; our protocol uses only additions and multiplications by public plaintext scalars, so it consumes a single level and never requires the expensive bootstrapping step [47] that refreshes a depleted budget.

A single-key scheme would require one party to hold the secret key and therefore be trusted with decryption. We instead use the multiparty variant of CKKS [48], in which the secret key is shared among the clients through a distributed key-generation protocol that produces a collective public key without ever materialising the secret in one place. Any party, including the server, may compute on ciphertexts under the collective public key, but decryption requires the cooperation of a threshold of clients, carried out as a collective key-switching protocol. No single party, and in particular not the server, can decrypt on its own. This is exactly the property our aggregation needs: the server performs the homomorphic computation on encrypted contributions, yet only the clients, acting together, can reveal the result.

D. The Shared Frame

The natural one-shot recipe (let each client train independently and have the server average the results) fails under heterogeneity. When clients begin from different initializations and fit different skewed distributions, their trained parameters occupy different regions of parameter space and their average lies in none of them: the updates point in conflicting directions and cancel when summed. The remedy is for every client to begin from the same point and move only a little, so their displacements live in one frame and add constructively rather than fighting one another.

A pretrained backbone provides this directly. Because the representation is fixed and shared, the trainable unit lives in the same parameter space for every client, and initializing it to a common public constant θ_0 places all clients at the same point in that space before any training. No data is consulted to build θ_0 and nothing is exchanged to agree on it; it is a fixed, public initializer. This is exactly the step a from-scratch federation cannot take and must replace with a data-dependent alignment phase, a phase that costs an extra round, exposes a per-client summary, and, when that summary is protected, spends a differential-privacy budget. Fine-tuning a pretrained model removes the phase, and with it the only channel through which client data would otherwise leave in the clear before encryption. We verify in section IV that this shared start is what makes the one-shot combination work (averaging independently trained units without it collapses), and that the frozen backbone supplies the start with no alignment phase.

The frame the backbone supplies is the representation, not the classifier. The shared initializer θ_0 is an untrained head: it places every client at the same point in parameter space but encodes nothing about any class. A client therefore contributes a useful displacement for a class only when it holds examples of that class, so the global model's accuracy on a class is bounded by how many clients cover it. This is the one accuracy cost of dropping alignment, and it is a property of *coverage* rather than of the frame. It is most visible when the label space is large and the skew extreme, so that each client sees only a sliver of the classes, and it narrows as each client sees more of the label space, closing well before the data become uniform. We characterize this gap directly in section IV, measured against an alignment-based reference that closes it only by exposing per-client class summaries in the clear, the exposure the cryptographic design exists to remove.

E. Local Fine-Tuning

Starting from θ_0, client j fine-tunes its trainable unit on its own data for K steps of ordinary supervised training, producing parameters $\theta_j^{(K)}$. The step budget K is deliberately small. This is not an optimisation shortcut but part of the method: a short trajectory keeps the trainable unit close to θ_0, and therefore inside the shared frame, so the displacement it produces is still expressed in the common coordinates. Allowed to train to convergence, each client would drift to its own local optimum and reintroduce exactly the conflict the shared start was meant to avoid.

At the end of its trajectory the client transmits not its parameters but the displacement

$$\Delta_j \;=\; \theta_j^{(K)} - \theta_0, \tag{1}$$

the distance it travelled from the shared start. Because every client departs from the same θ_0, these displacements are directly comparable across clients, which is what makes the linear combination that follows meaningful. A variant that distils a locally trained teacher into the unit, rather than fine-tuning on hard labels directly, fits the same template and is evaluated as an ablation in section IV.

F. Encrypted Linear Aggregation

The clients hold a joint encryption key produced by distributed key generation, so no single party, and in particular not the server, ever holds the secret key. Each client encrypts its displacement Δ_j; since the trainable unit is small relative to the backbone, this is a few tens of ciphertexts. The server fuses the per-client displacements into a single update of the shared model by the sample-weighted combination

$$\theta^\star \;=\; \theta_0 + \sum_{j=1}^{N} w_j\,\Delta_j, \qquad w_j = \frac{n_j}{\sum_i n_i}, \tag{2}$$

in which each client contributes in proportion to the data n_j behind it. Only the displacements are encrypted; the weights w_j and the initializer θ_0 are public and stay in plaintext. Forming eq. (2) therefore requires only multiplying each ciphertext by the public scalar w_j and adding the results, with θ_0 added in plaintext: there is no ciphertext-by-ciphertext multiplication, no encrypted optimizer state, and no bootstrapping, so the circuit has multiplicative depth one. A threshold of clients then jointly decrypts $\theta^\star$ by collective key-switching, and the server never observes a plaintext update. This is the central design choice: casting aggregation as an operation that is *linear in the encrypted quantities* yields a server computation that maps onto the least costly operations a levelled scheme provides, without the encrypted nonlinearities that make encrypted training expensive. The operations are also fully parallel (the clients fine-tune and encrypt independently, and the server's sum is an associative reduction), so the end-to-end wall-clock is one client's local computation plus a single aggregation, independent of N. We instantiate eq. (2) in a real multiparty

CKKS implementation and report its measured correctness and cost in section IV-F.

It is worth being precise about why this works, because eq. (2) is easily misread. Since the weights sum to one, the displacement form is algebraically a weighted average of the clients' fine-tuned units, $\theta^\star = \sum_j w_j \, \theta_j^{(K)}$. That linearity is what keeps the operation inexpensive to encrypt. But the same average over units trained independently, from different starts, to convergence is the naive combination that fails above. What makes the average land in a good region here is the protocol around it: every $\theta_j^{(K)}$ starts from the shared θ_0 and moves only a bounded distance, so all of them stay in the common frame and their weighted average is itself a sensible model. The contribution is not a new aggregation rule but a one-shot protocol under which a linear, and therefore inexpensively encryptable, aggregation produces a strong shared model.

G. Threat Model

The participants are N clients and a single aggregation server. The server is honest-but-curious: it follows the protocol but may try to infer private information from anything it observes. During aggregation it sees only ciphertexts under the collective key, and as established in section III-C it cannot decrypt them. A minority of clients may likewise be honest-but-curious and may collude, but any coalition below the decryption threshold learns nothing about another client's displacement, since recovering a plaintext requires the cooperation of the threshold. This is a stronger position than secure-aggregation protocols, where the revealed sum has itself been shown to leak across rounds [49], [50], because in our one-shot protocol the server never obtains a plaintext sum at all. Robustness to actively malicious or Byzantine clients is outside our scope; we discuss it as a direction for future work in section IV-J.

H. Security

The model updates are protected cryptographically: the aggregation of eq. (2) runs entirely under multiparty CKKS with threshold decryption, so the displacements Δ_j are never exposed to the server or to any minority of clients, and they are protected without any perturbation. The cryptographic layer adds no accuracy cost. This is the central privacy claim, and the point on which the method departs from differentially private one-shot federated learning, which injects noise into the updates it ultimately releases and so pays a utility tax on the very thing it ships.

We make this precise. Let the view of the honest-but-curious server be everything it receives: the public collective key, the public weights $\{w_j\}$ and initializer θ_0, the client ciphertexts $\{\mathsf{Enc}(\Delta_j)\}_{j=1}^N$, and the homomorphically computed $\mathsf{Enc}(\theta^\star)$. We write $\mathsf{view}_{\mathrm{srv}}(\{\mathcal{D}_j\})$ for this view as a function of the private datasets.

Proposition 1. *Under the IND-CPA security of the multiparty CKKS scheme and its t-out-of-N threshold decryption, for any honest-but-curious server colluding with up to $t-1$ clients there is a probabilistic polynomial-time simulator $\mathcal{S}$, given only the public parameters, such that $\mathsf{view}_{\mathrm{srv}}(\{\mathcal{D}_j\}) \overset{c}{\approx} \mathcal{S}$. In particular the server's view is computationally independent of the private datasets $\{\mathcal{D}_j\}$ and the displacements $\{\Delta_j\}$, and the server cannot recover any plaintext, including $\theta^\star$, without a decryption quorum.*

Proof sketch. The simulator outputs the public parameters together with N encryptions of zero under the collective key. By IND-CPA each genuine $\mathsf{Enc}(\Delta_j)$ is computationally indistinguishable from $\mathsf{Enc}(0)$; a hybrid over the N ciphertexts replaces them one at a time, so the whole collection is indistinguishable from the simulated one and carries no information about $\{\Delta_j\}$ or the data they derive from. The aggregate $\mathsf{Enc}(\theta^\star)$ is a deterministic function of those ciphertexts and the public $\{w_j\}, \theta_0$, so it adds nothing. Decryption needs t shares of a secret key that is never reconstructed in one place; the server with at most $t-1$ clients holds too few shares to decrypt. □

Proposition 1 pins down where information can flow. Because there is no alignment phase, no data-derived quantity leaves a client before encryption, so the protocol exposes private information through exactly one channel, intrinsic to the task rather than an artifact of our construction: the released model itself. After threshold decryption every client holds $\theta^\star$ in the clear, and white-box access to the jointly built model is unavoidable in any protocol whose purpose is to hand the parties a shared model. The residual information that model carries about training data is what we quantify with membership inference in section IV-G.

IV. Experiments

We evaluate the protocol's central claim: that encryption protects each client's contribution at no cost to the shared model's accuracy, where one-shot differentially private federated learning must trade accuracy for the same protection. The evaluation tests that claim and the design choices behind it. We ask, in order: what does fine-tuning the pretrained layers add over a frozen-feature head (the choice that makes a single linear encrypted aggregation work); does the shared model hold up across heterogeneity and client count; what do the bounded trajectory and its scaling coefficient contribute; and what does the cryptographic protocol cost in time, communication, and residual leakage. Throughout, the question is not whether the global model attains the highest possible test accuracy, but whether the clients jointly obtain a single model close to a centrally fine-tuned one, in one round, with cryptographic privacy that adds no accuracy cost.

A. Setup

a) No public data.: Every client starts from the *same* public pretrained backbone with a zero-initialized low-rank adapter and a freshly initialized head, the standard adapter initialization θ_0, which carries no client data and classifies at chance. Each client fine-tunes the adapter and head on its own private data for a bounded K steps and uploads the encrypted displacement; the server forms the depth-one weighted sum of eq. (2). No labelled public set is used anywhere; section IV-E

TABLE II
EVALUATION AXES. EACH VARIES ONE DIMENSION AROUND THE DEFAULT ($N = 10$, RANK-8 ADAPTER, $K = 200$, THREE SEEDS); NO PUBLIC DATA IS USED ANYWHERE.

Axis	Setting	Question it probes
Trainable unit	head-only vs. rank-$\{4, 8, 16\}$ adapter	value of fine-tuning pretrained layers
Heterogeneity α	0.05 (severe) to 1.0 (mild)	robustness to label skew
Clients N	10, 20, 50, 100	scalability in participants
Trajectory K	100, 200, 400	the bounded-step budget

discusses the easier case in which a deployment does have one.

b) Tasks and backbones.: We evaluate on four text classification tasks of increasing label-space size: AG-News (4 classes), TREC (6), DBpedia (14), and Banking77 (77), with two frozen encoders, RoBERTa-base and MPNet, and on CIFAR-100 with a ViT vision backbone. Data are partitioned across clients by a Dirichlet distribution over labels with concentration α (the standard label-skew partition: a smaller α leaves each client a few dominant classes and almost none of the rest). Unless stated otherwise there are $N = 10$ clients, the adapter is rank 8, the trajectory is $K = 200$ steps, and every number is averaged over three seeds.

c) What we report.: We report the test accuracy of the HE-IFD global model; the shared start θ_0 classifies at chance, since it carries no task head. Two reference points frame it: a frozen-feature head (a linear probe trained under the same protocol) and, as the ceiling, the same model fine-tuned centrally on the pooled private data, what one party would obtain holding all of it. The two quantities of interest are the *increment* of HE-IFD over the frozen-feature head and the *gap* below the centralized ceiling; the centralized accuracy itself we give in the relevant captions rather than repeat in every row. Table II lists the axes.

B. The Value of Fine-Tuning the Pretrained Layers

The design rests on adapting the backbone's own layers, not training a head on frozen features: a low-rank adapter is small enough to encrypt inexpensively yet fine-tunes the pretrained representation itself. The first experiment measures what that adds. Figure 2 and table III compare, at $N = 10$ and moderate heterogeneity, the global model produced with a frozen-feature head against the one produced by adapter fine-tuning, against the centralized ceiling.

Across the four text tasks, HE-IFD adds between 0.14 and 0.37 accuracy over the frozen-feature head (a linear probe), well above the chance start; because only the small adapter and head enter the encrypted combination, the protocol and its cost do not change across tasks. On CIFAR-100 the frozen ViT already gives a strong linear-probe head, leaving little room for the adapter to add, so the value of fine-tuning the layers is largest where the frozen features are weak. The remaining gap to the centralized fine-tune tracks the size of the label space: it is 0.20 on the 14-class DBpedia and widens to 0.52 on the 77-class Banking77, where a single round must cover many classes each client barely observes. HE-IFD produces the model in one round, from a start that classifies at chance, with no client exposing its data and with encryption that perturbs nothing.

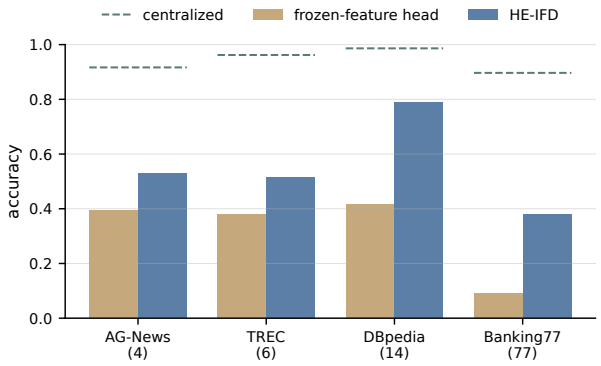

Fig. 2. Fine-tuning the pretrained layers, not a frozen-feature head, lifts the single encrypted aggregation well above the chance start and the linear probe; the gap to centralized grows with the size of the label space, and no client data is shared.

TABLE III
FINE-TUNING THE PRETRAINED LAYERS VS. A FROZEN-FEATURE HEAD ($N = 10$, $\alpha = 0.1$, RANK-8 ADAPTER, THREE SEEDS). HE-IFD IS THE RANK-8 LORA MODEL; THE HEAD IS A LINEAR PROBE; THE INCREMENT IS HE-IFD'S LIFT OVER THE HEAD; $\pm$ IS THE SEED STANDARD DEVIATION. THE CENTRALIZED FINE-TUNE REACHES 0.88–0.99, THE CEILING THE GAP IS MEASURED TO.

Task (C)	head	HE-IFD	increment	gap
AG-News (4)	0.39	0.53±0.21	+0.14	0.39
TREC (6)	0.38	0.52±0.21	+0.14	0.45
DBpedia (14)	0.42	0.79±0.02	+0.37	0.20
Banking77 (77)	0.09	0.38±0.01	+0.29	0.52
CIFAR-100 (100)	0.54	0.53±0.02	−0.01	0.35

C. Robustness to Heterogeneity and Client Count

Two properties the one-shot aggregation depends on are that it survives label skew and that it does not destabilise as clients are added. Figure 3 and table IV report both on DBpedia. Heterogeneity is the harder axis: as α falls each client sees fewer classes, and HE-IFD's accuracy declines monotonically from 0.98 at $\alpha = 1.0$ (within 0.003 of the centralized 0.99) to 0.58 at $\alpha = 0.05$, while the centralized fine-tune barely moves. The gap a single linear aggregation pays is therefore a property of heterogeneity: near zero when the data are close to uniform, and 0.41 only under severe skew, which we return to in section IV-J. Across client count the aggregation does not degrade: HE-IFD reaches 0.79 at $N = 10$ and holds at 0.85–0.86 from $N = 20$ to $N = 100$ (single seed beyond $N = 10$), so adding participants does not destabilise the single aggregation, and the protocol's cost (section IV-F) is the only quantity that grows with N.

D. Trajectory Length

The trajectory length sets how far each client moves from the shared start before its displacement is aggregated. Table V

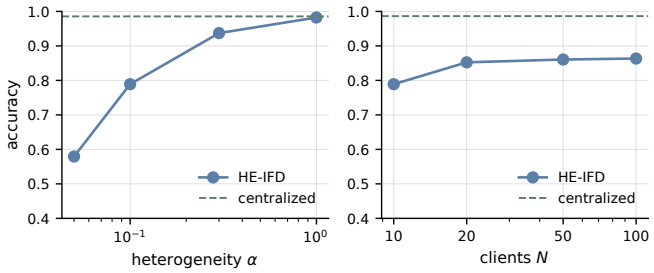

Fig. 3. HE-IFD holds up across client count and degrades gracefully with label skew, the gap to the centralized fine-tune (≈ 0.99) widening only at the most severe heterogeneity.

TABLE IV
HE-IFD ACCURACY ACROSS HETEROGENEITY AND CLIENT COUNT (DBPEDIA, RANK-8 ADAPTER, THREE SEEDS); THE CENTRALIZED FINE-TUNE IS ≈ 0.98 THROUGHOUT.

Heterogeneity ($N = 10$)		Clients ($\alpha = 0.1$)	
α	HE-IFD	N	HE-IFD
0.05	0.58	10	0.79
0.10	0.79	20	0.85
0.30	0.94	50	0.86
1.00	0.98	100	0.86

varies the step budget K on DBpedia: within the swept range a longer trajectory helps monotonically, lifting HE-IFD from 0.68 at $K = 100$ to 0.87 at $K = 400$ and narrowing the gap to centralized to 0.12, the frozen backbone keeping even the longest trajectory's displacements coherent enough to sum.

E. When Public Data Is Available

We study the harder setting in which no public data exists, because that is where a one-shot encrypted aggregation is actually needed. When a deployment does have a public labelled set, the problem is strictly easier and standard recipes apply directly: the shared initialization can be warm-started on it, or a model can be trained on the public data and then refined. With enough of it, training a model from scratch on the public set is itself an option. We therefore treat public data as outside the method and rely on none of it anywhere in this paper.

F. Homomorphic-Encryption Implementation and Cost

We do not simulate the cryptography. We implement the entire protocol end to end in real multiparty CKKS on the Lattigo library [54], [48]: distributed key generation, encryption under the joint public key, the homomorphic weighted aggregation of eq. (2), and threshold decryption by a collective key-switch, and the decrypted aggregate matches the plaintext computation to within CKKS encoding precision (verified below). The server's only cryptographic operation is that depth-one weighted sum, and its cost is set entirely by the number of *fine-tuned* parameters that enter a ciphertext, not by the frozen backbone behind them. We measure the per-operation rates directly and calculate every reported configuration from them, so a small adapter and head are inexpensive to encrypt whatever model they sit on.

TABLE V
TRAJECTORY LENGTH: HE-IFD ACCURACY (DBPEDIA, $N = 10$, $\alpha = 0.1$, THREE SEEDS); CENTRALIZED ≈ 0.99.

K	HE-IFD	gap
100	0.68	0.30
200	0.76	0.22
400	0.87	0.12

TABLE VI
PER-CLIENT COMMUNICATION PER ROUND (MULTIPARTY CKKS, RING 2^{14}, 0.5 MIB PER CIPHERTEXT). A ROUND IS ONE UPLOAD AND ONE DOWNLOAD; TOTALS SCALE LINEARLY IN N (FIG. 4).

Trainable unit	ciphertexts	up=down per client
LoRA adapter+head	**38**	19 **MiB**
text head (4-class)	1	0.50 MiB
vision head (100-class)	10	5.0 MiB

a) Parameters and setup.: All figures use ring degree 2^{14} (8192 slots per ciphertext), a modulus chain $\log Q = \{55, 45\}$ with key-switch prime $\log P = \{61\}$, and scale 2^{45}. The circuit is multiplicative *depth one* (one plaintext-scalar multiplication and a rescale, with no relinearization and no bootstrapping), and decryption is N-out-of-N (no single party can decrypt). Communication figures are exact; timings are single-run wall-clock on one core of a commodity laptop CPU (Apple M4) and are reported as indicative. The circuit uses no ciphertext rotations and no relinearization, and because the only server operation is the one-shot aggregation, there is no encrypted training loop and hence no convergence behaviour to report under encryption. The server accumulates the weighted sum as a streaming reduction, holding at most two adapter-sized ciphertext sets at once (about 38 MiB), so its memory does not grow with N.

b) Communication.: A round is one upload and one download, and the protocol completes in a single round. The per-client traffic is fixed by the trainable-parameter count: a unit of up to 8192 parameters is a single 0.5 MiB ciphertext. The headline configuration, a rank-8 LoRA adapter with a classifier head (about 3.1×10^5 parameters), occupies 38 ciphertexts, or 19 MiB per client; freezing the adapter's projection halves this to about 10 MiB. Total traffic scales linearly in the number of clients (table VI). The contrast with encrypted full-model schemes is structural: on the same backbone, encrypting the full RoBERTa-base (1.25×10^8 parameters) would move about 7.5 GiB per client *per round*, and such schemes run for many rounds, where HE-IFD encrypts only the adapter and runs once (fig. 4).

c) Computation.: The server's homomorphic work is milliseconds at deployment scale and stays sub-second even at $N = 100$ on the largest head (table VII). The measured operations follow simple laws: a client encrypts in about 2.2 ms per ciphertext (in parallel across clients), while the server aggregation and the threshold decryption scale as ≈ 0.39 and ≈ 0.23 ms per $N \times$ (ciphertexts/client), and the one-time distributed key generation grows linearly in N at about 1 ms per party. From these rates the rank-8 adapter (38 ciphertexts) costs about 84 ms to encrypt per client and, at $N = 100$, about

TABLE VII
COMPUTATION TIME (MS, ONE CPU CORE). DKG IS A ONE-TIME SETUP; THE ENCRYPT FIGURE IS PER CLIENT (CLIENTS ENCRYPT IN PARALLEL); AGGREGATION AND DECRYPTION SCALE LINEARLY IN $N\times$CIPHERTEXTS.

Trainable unit	N	DKG	encrypt/client	aggregate	decrypt
LoRA adapter	10	13	84	148	87
LoRA adapter	100	96	84	1482	874
text head	10	13	2.3	3.9	2.4
text head	100	96	2.2	38.9	23.2
vision head	10	11	21.5	39.0	24.6
vision head	100	98	21.7	382.2	216.5

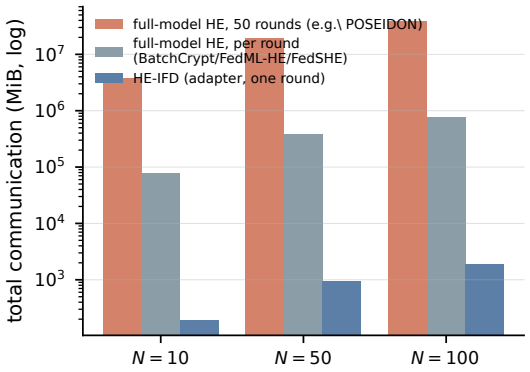

Fig. 4. Total communication on a shared backbone (RoBERTa-base), calculated for CKKS ring 2^{14} (0.5 MiB per ciphertext; 50 rounds for the iterative schemes). HE-IFD encrypts only the adapter and runs once; full-model encrypted schemes move gigabytes per round (BatchCrypt, FedML-HE, FedSHE) to terabytes over training (POSEIDON-style encrypted training).

1.5 s to aggregate and 0.9 s to decrypt (table VII). There is no bootstrapping anywhere in the protocol.

d) Comparison with other encrypted schemes.: Table VIII places HE-IFD against the encrypted federated-learning approaches. Two properties separate it from all of them. First, it is one-shot: it encrypts a single contribution and runs one round, where encrypted-training schemes such as POSEIDON [4] run multiparty CKKS for many rounds, hours of wall-clock even on small models, and aggregation schemes such as BatchCrypt, FedML-HE, and FedSHE [6], [7], [8] encrypt and sum updates on every round of an iterative protocol. Second, it encrypts only the small adapter and head, not the full model: the per-client object is 19 MiB against the gigabytes a full-model scheme moves on the same backbone, and there is no bootstrapping because the circuit is depth one. The secure-aggregation primitive of [5] is one-round but protects a per-round sum within an iterative protocol rather than a one-shot contribution. Figure 4 shows the per-round communication gap on a shared backbone, before the round-count multiplier that further separates a one-round protocol from a multi-round one.

The encrypted object stays depth one and the server cost grows only with the ciphertext count, never with the size of the frozen backbone the adapter sits in.

e) Correctness.: The decrypted aggregate matches the plaintext computation to a relative ℓ_2 error between 1.4×10^{-9} ($N = 5$) and 2.6×10^{-8} ($N = 100$). This is the encoding precision of CKKS at scale 2^{45}; it grows mildly with N as ciphertext additions and the key-switch smudging noise accumulate, and it stays several orders of magnitude below any tolerance the decoded model is sensitive to. Because the circuit is depth one, the error is not amplified.

f) Comparison with prior cryptographic federated learning.: The contrast with prior homomorphic federated learning is structural, not incremental. Encrypted-training methods such as POSEIDON [4] reach comparable accuracy but through many rounds of homomorphic computation that the source paper reports at roughly 1.4 hours of wall-clock; secure-aggregation primitives [5] sum updates once per round inside an iterative protocol. Our protocol runs in a single round whose server computation is milliseconds and whose communication is a few megabytes, because the only encrypted object is a bounded fine-tuning displacement of a small adapter, combined at multiplicative depth one. The broader method-level comparison is in table IX.

G. Residual Leakage

The cryptographic guarantee of section III-H covers the contributions, not the final model, which every party obtains in the clear after decryption. This is not a gap a stronger mechanism could close: any protocol that hands separate parties a shared, useful model must move knowledge between them, and a model that classifies categories a client never saw locally necessarily encodes information about the data that taught it those categories. The meaningful question is not whether residual leakage is zero, which it cannot be, but how large an attack surface a protocol exposes, and our design makes that surface as small as a one-shot federated method can. There is no alignment phase, so nothing data-derived is released before encryption, and the only object exposed in the clear is the single decrypted model.

Read as attack surfaces, the alternatives differ by orders of magnitude. Multi-round federated learning publishes a fresh update every round, so a curious server or peer accumulates a long sequence of views of each client over training [31], [32]. One-shot methods that rely on differential privacy ship, in a single object, essentially the whole of a client's local knowledge: a synthetic copy of its data, or a noised model. HE-IFD exposes only the final aggregate: the per-client displacements are encrypted and never appear in the clear, and because there is no alignment phase there is no prototype channel either.

We measure the residual on that one open channel with membership-inference attacks [33], [34], [35], including the attacks tailored to the federated setting [40], [41], [43], [42], querying the released model both as an external adversary and as a fellow client that brings its own data as a prior. The leakage stays small: on the language task the strongest shadow-model attack remains at the level of random guessing across heterogeneity levels, so a participant learns essentially nothing about another's membership from the shared model beyond what any usable classifier of that data would reveal.

We do not claim the released model leaks nothing. A model that gives every client coverage of classes it never observed must carry information about the data that supplied that coverage, and an adversary with strong priors about a

TABLE VIII
HE-IFD AGAINST ENCRYPTED FEDERATED-LEARNING SCHEMES. COMMUNICATION IS PER CLIENT; HE-IFD'S IS CALCULATED FOR A RANK-8 ADAPTER (ROBERTA-BASE, 38 CIPHERTEXTS), THE FULL-MODEL FIGURE FOR ENCRYPTING THE SAME BACKBONE.

Method	crypto	encrypted object	rounds	comm/client
HE-IFD (ours)	multiparty CKKS	adapter+head	**1**	19 **MiB**
POSEIDON	multiparty CKKS	full model	many	GiB ($\sim$1.4 h)
BatchCrypt / FedML-HE / FedSHE	CKKS	full model	many	GiB / round
Hyb-Agg	multi-key CKKS	update sum	per round	secure sum

client's distribution can recover some of it; this is a property of useful shared models, not of our protocol in particular. What the protocol guarantees is that this residual is the *entire* surface (not multiplied across the rounds of an iterative protocol, and not handed over wholesale as released data), and driving the exposed surface down to that inherent floor, rather than promising to remove a leak that cannot be removed when parties set out to build a common model, is the privacy goal the design meets.

H. Comparison with Prior Work

Table IX places the method among representative federated-learning approaches along the axes that matter here: whether the protocol is genuinely one-shot, what kind of privacy it provides on the contributions, how many rounds it needs, and its principal limitation relative to our goal. The one-shot distillation methods are either plaintext, and so offer no privacy on the contributions, or differentially private, and so pay a utility cost on the released model. The homomorphic-encryption methods either run for many rounds, as in encrypted training, or are secure-aggregation primitives that sum updates over an iterative protocol. The combination we occupy (a one-shot federated fine-tuning whose aggregation is protected by homomorphic encryption that adds no accuracy cost) has, to our knowledge, not been demonstrated before.

Two comparisons are the informative ones. Against POSEIDON, the closest cryptographic peer, the privacy guarantee is the same kind (encryption with no noise), but POSEIDON reaches its accuracy through multi-round encrypted *training* that the source paper reports at roughly 1.4 hours, whereas our protocol completes in a single round whose server computation is milliseconds and whose communication is tens of megabytes (section IV-F). Against the differentially private one-shot methods, the natural utility peers, the contrast is the central one: their accuracy is obtained under a privacy budget that perturbs the released model, whereas encryption introduces no such perturbation.

[†]FuseFL's communication equals one-shot but it runs in K block-wise rounds.

I. Beyond Pretrained Backbones

The protocol does not require a pretrained backbone; it requires a shared frame. Where clients instead train a small network from a common random initialization, the same one-shot encrypted aggregation applies, but the shared frame must be established rather than inherited. Without a pretrained representation the clients' bounded updates are expressed in a coordinate system fixed only by the random seed, so agreeing on a useful starting point reintroduces the kind of alignment step a pretrained backbone removes. We therefore present pretrained fine-tuning as the primary setting and treat the from-scratch variant as a generalization whose alignment cost is left to future work.

J. Scope and Limitations

Two boundaries follow from well-understood properties rather than incidental shortcomings. First, the single linear aggregation pays a gap to centralized fine-tuning that widens under severe label skew, because a client contributes signal only for classes it holds and the shared head is untrained; this is an inherent property of combining bounded one-shot updates without an alignment phase, visible above as the $\alpha = 0.05$ gap, and the bounded trajectory contains it rather than removing it. Second, the method fine-tunes a pretrained backbone, so it inherits that backbone's fit to the task and applies to the label-groundable foundation models used for transfer today rather than to a backbone with no relevant pretraining. Both are stated where they apply; neither touches the privacy claim, which holds regardless of the accuracy the shared model reaches.

a) Malicious and colluding clients.: Our guarantees assume honest-but-curious participants. An actively malicious client could upload a crafted displacement to bias the shared model, and because the contributions are encrypted the server cannot inspect them, so plaintext outlier filtering does not apply. Two defences are compatible with the encryption and are left to future work: encrypted-domain robust aggregation, which replaces the sample-weighted sum with a coordinate-wise median or trimmed mean evaluated homomorphically, and an upload-time norm bound enforced by a zero-knowledge proof, which caps any single client's influence without revealing its update. The one-shot structure shrinks the attack surface to a single round, but it also removes the cross-round consistency checks a multi-round protocol could use, and reconciling robust aggregation with depth-one encryption is the open problem this direction must solve.

V. CONCLUSION

We presented HE-IFD, a one-shot federated fine-tuning in which the server's only operation on client contributions is a linear aggregation computed under multiparty homomorphic encryption at multiplicative depth one. The construction gives the contributions cryptographic privacy at no accuracy cost and at the communication budget of a single round; because the backbone is frozen and stays outside the encrypted combination, the protocol and its cost are independent of the

TABLE IX
COMPARISON WITH REPRESENTATIVE FEDERATED-LEARNING METHODS. REPORTED ACCURACIES ARE TAKEN VERBATIM FROM EACH SOURCE PAPER AT *its own* SETUP, SO THE COLUMN IS CONTEXT, NOT A LIKE-FOR-LIKE RANKING; THE PROTOCOL AXES (ONE-SHOT, PRIVACY, ROUNDS) ARE THE CONTROLLED COMPARISON. OURS IS THE ONLY METHOD THAT IS SIMULTANEOUSLY ONE-SHOT, FINE-TUNING-BASED, AND PROTECTED BY ENCRYPTION THAT ADDS NO ACCURACY COST.

Method	One-shot	Privacy	Rounds	Reported acc. (own setup)	Principal limitation vs. our goal
FedDF [18]	no	none	$\sim$100	rounds-to-target (CIFAR-10)	many rounds; no privacy
FuseFL [21]	no†	none	K blocks	84.3% (CIFAR-10)	K rounds; no privacy
DENSE [19]	yes	none	1	data-free (CIFAR-10)	no privacy on contributions
FedLPA [22]	yes	none	1	85.8% (MNIST)	no privacy; needs Fisher info
FedKT [26]	yes	DP-capable	1	90.5% (MNIST)	lossy when private
FedAUXfdp [24]	yes	DP (lossy)	1	75.2% (CIFAR-10, ε=0.5)	accuracy falls with budget
FedDiff [25]	yes	DP (lossy)	1	27.8% (CIFAR-10, ε=10)	accuracy falls with budget
POSEIDON [4]	no	HE (no noise)	many	89.9% (MNIST, $\sim$1.4 h)	multi-round; encrypted training
HE secure agg. [5]	no	HE	per round	protocol only	aggregation, not fine-tuning
HE-IFD (ours)	**yes**	**HE (no noise)**	**1**	**0.79/0.53 (DBpedia/AG-News)**	**none**

model behind it. What makes a one-shot linear aggregation succeed is that every client begins from the same public pretrained backbone and a standard adapter initialization, so the bounded displacements they upload are expressed in a common frame and add constructively, with no public data, no alignment phase, and no data-derived quantity leaving a client before encryption. Across vision and language tasks the method yields a common model stronger than any client's own and, outside the most skewed regimes, close to a centrally fine-tuned one, and a multiparty CKKS implementation reproduces the encrypted aggregation within the scheme's approximation bounds at a few megabytes per round. Extending the protocol to actively malicious participants, where verifiable homomorphic encryption [55], [56] could certify that the server computed the aggregation honestly, and tightening the analysis of what the released model itself reveals, are natural directions for future work.

ACKNOWLEDGMENTS

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude Sonnet 4.6 and Claude Opus 4.6 to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[2] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *ACM CCS*, 2017, pp. 1175–1191.

[3] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 308–318. [Online]. Available: https://arxiv.org/abs/1607.00133

[4] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.

[5] Kemmaka and Tran, "Hyb-Agg: Communication-efficient secure aggregation via hybrid multi-key homomorphic encryption and masking," *arXiv:2511.23252*, 2025.

[6] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning," in *USENIX ATC*. USENIX Association, 2020, pp. 493–506.

[7] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, "Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system," in *NeurIPS*, vol. 36, 2023.

[8] X. Wei, R. Huang, L. Lei, and J. Wu, "Fedshe–cq: A communication-efficient homomorphic encryption framework for federated learning," *Computer Networks*, vol. 260, p. 111813, 2025.

[9] K. Garimella, N. K. Jha, and B. Reagen, "Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning," in *PPML Workshop, CCS*, 2021.

[10] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkowich, "A methodology for training homomorphic encryption friendly neural networks," in *ACNS Workshops*, ser. Lecture Notes in Computer Science, vol. 13285. Springer, 2022, pp. 536–553.

[11] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 7, 2025.

[12] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.

[13] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *Privacy in Machine Learning Workshop (PriML), NeurIPS*, 2021. [Online]. Available: https://www.eurecom.fr/en/publication/5626

[14] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019. [Online]. Available: https://arxiv.org/abs/1902.11175

[15] J. Wang *et al.*, "Towards one-shot federated learning: Advances, challenges, and future directions," *arXiv preprint arXiv:2505.02426*, 2025.

[16] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019. [Online]. Available: https://arxiv.org/abs/1910.03581

[17] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "DS-FL: Distillation-based semi-supervised federated learning for medical image classification," in *IEEE ICC*, 2021.

[18] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *NeurIPS*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/18df51b97ccd68128e994804f3eccc87-Abstract.html

[19] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, vol. 35, 2022, pp. 21 414–21 428. [Online]. Available: https://arxiv.org/abs/2112.12371

[20] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.

[21] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.

[22] X. Liu, L. Liu, F. Ye, Y. Shen, X. Li, L. Jiang, and J. Li, "FedLPA: One-shot federated learning with layer-wise posterior aggregation," in *NeurIPS*, 2024.

[23] Y. Zhang *et al.*, "Federated one-shot learning with data-free distillation," in *NeurIPS*, 2024.

[24] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.

[25] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025, pp. 2601–2610. [Online]. Available: https://openaccess.thecvf.com/content/WACV2025/html/Mendieta_Navigating_Heterogeneity_and_Privacy_in_One-Shot_Federated_Learning_with_Diffusion_Models_WACV_2025_paper.html

[26] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021, pp. 1484–1490.

[27] L. Sun and L. Lyu, "Federated model distillation with noise-free differential privacy," in *IJCAI*, 2021, pp. 1563–1569.

[28] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, vol. 32, 2019.

[29] D. I. Dimitrov, M. Baader, and M. Vechev, "SPEAR: Exact gradient inversion of batches in federated learning," in *NeurIPS*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/c13cd7feab4beb1a27981e19e2455916-Abstract-Conference.html

[30] V. Carletti, P. Foggia, C. Mazzocca, G. Parrella, and M. Vento, "SoK: Gradient inversion attacks in federated learning," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/carletti

[31] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE S&P*, 2017, pp. 3–18. [Online]. Available: https://arxiv.org/abs/1610.05820

[32] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019, pp. 739–753.

[33] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *31st IEEE Computer Security Foundations Symposium (CSF)*, 2018, pp. 268–282. [Online]. Available: https://arxiv.org/abs/1709.01604

[34] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: https://arxiv.org/abs/1806.01246

[35] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *IEEE S&P*, 2022, pp. 1897–1914. [Online]. Available: https://arxiv.org/abs/2112.03570

[36] R. Kerkouche, G. Mema, M. Fritz, J. Ramon, and N. Samarin, "Client-specific property inference against secure aggregation in federated learning," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2303.03908

[37] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 1322–1333. [Online]. Available: https://dl.acm.org/doi/10.1145/2810103.2813677

[38] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017, pp. 603–618.

[39] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023, pp. 12 198–12 207.

[40] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, ser. Lecture Notes in Computer Science, vol. 13423. Springer, 2023, pp. 364–378.

[41] X. Wang, L. Wu, and Z. Guan, "GradDiff: Gradient-based membership inference attacks against federated distillation with differential comparison," *Information Sciences*, vol. 658, p. 120068, 2024.

[42] M. Jagielski, M. Nasr, K. Lee, C. Choquette-Choo, N. Carlini, and F. Tramèr, "Students parrot their teachers: Membership inference on model distillation," in *NeurIPS*, vol. 36, 2023.

[43] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023. [Online]. Available: https://dl.acm.org/doi/10.1016/j.jisa.2023.103475

[44] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *PoPETs*, vol. 2025, no. 4, pp. 201–215, 2025.

[45] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017, pp. 409–437.

[46] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.

[47] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10820. Springer, 2018, pp. 360–384. [Online]. Available: https://eprint.iacr.org/2018/153

[48] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.

[49] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, vol. 37, no. 8, 2023, pp. 9925–9933.

[50] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*. ACM, 2023, pp. 15–30.

[51] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1708.07747

[52] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[53] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *ICLR*, 2023.

[54] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.

[55] A. Viand, C. Knabenhans, and A. Hithnawi, "SoK: Fully homomorphic encryption compilers and verifiable computation," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2301.07041

[56] S. Atapoor, K. Baghery, H. V. L. Pereira, and J. Spiessens, "Verifiable FHE via lattice-based SNARKs," Cryptology ePrint Archive, Paper 2024/582, 2024. [Online]. Available: https://eprint.iacr.org/2024/582

HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.

Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.

Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. He is an ACM and IEEE Senior Member, and a co-founder of Xtinge Technology Inc.